Agenta Coolify Deployment Guide

Prepared: May 15, 2026

What This Repo Contains

This repository is a small deployment wrapper for Agenta OSS. It does not rebuild Agenta from source. It pulls Agenta's official GHCR images and runs the required web, API, worker, services, Postgres, Redis, SuperTokens, and Nginx containers.

Public traffic should go only to the `nginx` service. Nginx routes `/` to the web UI, `/api/` to the API, and `/services/` to Agenta services.

Why This Shape Works Well In Coolify

Agenta's upstream quick start uses a Docker Compose file in `hosting/docker-compose/oss/docker-compose.gh.yml` and publishes GHCR images. Coolify also supports Docker Compose directly, where the compose file is the source of truth and you assign a domain to the service that should be public.

This repo keeps that model, but replaces the upstream internal Traefik service with a static Nginx router. That avoids mounting `/var/run/docker.sock` inside the application stack and leaves TLS/domain routing to Coolify's own proxy.

Files To Know

- `docker-compose.yml`: production compose stack for Coolify.
- `docker-compose.local.yml`: optional local port mapping for `localhost:8080`.
- `.env.example`: environment variables to copy into Coolify.
- `nginx/nginx.conf`: internal route map for web, API, and services.
- `postgres/init-db-oss.sql`: database bootstrap used on the first Postgres start.
- `scripts/generate-secrets.sh`: helper for generating production secrets.

Prerequisites

- A Coolify server with Docker working.
- A DNS record pointing your desired hostname to the Coolify server.
- A Git repo containing this folder.
- Enough memory for Agenta plus Postgres and Redis. Start with at least 4 GB RAM if possible.

Step 1: Create Your Git Repo

From this folder:

```
git init
git add .
git commit -m "Add Agenta Coolify deployment"
git remote add origin <your-repo-url>
git push -u origin main
```

If this folder already has a remote after you publish it, use your normal push workflow instead.

Step 2: Generate Secrets

Run:

```
./scripts/generate-secrets.sh
```

Copy the output. You will paste those values into Coolify environment variables.

Step 3: Create The Coolify Resource

In Coolify:

1. Create a new Project or choose an existing one.
2. Add a new Resource.
3. Choose your Git provider/repository.
4. Select Docker Compose as the build/deployment type.
5. Use `docker-compose.yml` as the compose file.
6. Save and let Coolify parse the services.

Step 4: Assign The Domain

Assign your public domain to the `nginx` service. The service listens on container port `80`, so use:

```
https://agenta.example.com
```

If Coolify asks for the service port explicitly, use port `80`.

Step 5: Add Environment Variables

In Coolify's environment variable screen, add the values from `.env.example`.

Required production values:

```
AGENTA_WEB_URL=https://agenta.example.com
AGENTA_API_URL=https://agenta.example.com/api
AGENTA_SERVICES_URL=https://agenta.example.com/services
AGENTA_ALLOWED_DOMAINS=agenta.example.com
```

```
AGENTA_AUTH_KEY=<generated>
AGENTA_CRYPT_KEY=<generated>
SUPERTOKENS_API_KEY=<generated>
POSTGRES_PASSWORD=<generated>
AGENTA_TELEMETRY_ENABLED=false
```

Replace `agenta.example.com` with your real domain.

Optional LLM provider keys:

```
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
GEMINI_API_KEY=
GROQ_API_KEY=
MISTRAL_API_KEY=
OPENROUTER_API_KEY=
```

Step 6: Deploy

Click Deploy in Coolify. The first boot can take a few minutes because Postgres initializes and Alembic migrations run before the API and workers settle.

Expected persistent volumes:

- `postgres-data`
- `redis-volatile-data`
- `redis-durable-data`

Step 7: Verify

Open your domain:

```
https://agenta.example.com
```

Then check:

```
https://agenta.example.com/api/health
```

If the UI loads but API calls fail, confirm that `AGENTA_API_URL` exactly matches your public domain plus `/api`.

Local Smoke Test

For a local test on your machine:

```
cp .env.example .env
docker compose --env-file .env -f docker-compose.yml -f docker-compose.local.yml up -d
```

Open:

```
http://localhost:8080
```

Stop it with:

```
docker compose --env-file .env -f docker-compose.yml -f docker-compose.local.yml down
```

Upgrades

The repo defaults to `latest` image tags. For safer production upgrades, pin the image tags:

```
AGENTA_WEB_IMAGE_TAG=v0.99.9
AGENTA_API_IMAGE_TAG=v0.99.9
AGENTA_SERVICES_IMAGE_TAG=v0.99.9
```

Then update intentionally, redeploy, and watch the `alembic`, `api`, and worker logs.

Backups

Back up the Postgres volume before upgrades or major configuration changes. At minimum, use `pg_dump` from the `postgres` container or Coolify's backup tooling if available.

Redis durable data is also persisted, but Postgres is the critical source of truth.

Security Notes

• Keep Postgres and Redis private. This compose file does not publish them to host ports.

• Do not commit real `.env` files or generated secrets.

• Rotate provider API keys from the provider dashboard if they are ever exposed.

• Keep `POSTGRES_PASSWORD` stable after first deployment unless you also rotate the database user's existing password.

References

• Agenta GitHub: https://github.com/Agenta-AI/agenta

• Agenta docs: https://docs.agenta.ai/

• Coolify Docker Compose docs: https://coolify.io/docs/knowledge-base/docker/compose